ENSA Marrakech

Université Cadi Ayyad

Génie Cyber-Défense et Systèmes de Télécommunications Embarqués

Projet de fin de semestre – DevSecOps & GRC

CAHIER DES CHARGES

Plateforme DevSecOps sur Kubernetes avec démarche GRC

Application de gestion de tâches

Python (FastAPI) • Next.js • MongoDB • AWS EKS

Réalisé par :

Sayf Eddine LAAMRI

Encadré par :

Mr. Atlas

6 novembre 2026

Table des matières

1 Introduction et contexte du projet **1**

2 L'application : un gestionnaire de tâches très simple **2**

2.1 Diagramme pieuvre 2

3 Architecture globale **3**

4 Diagrammes UML **4**

4.1 Cas d'utilisation 4

4.2 Diagramme de composants 4

4.3 Diagramme de classes 4

4.4 Diagramme de séquence : ajouter une tâche 5

5 Architecture de la base de données **5**

6 Outils utilisés **6**

7 Sécurité et démarche GRC **6**

8 Planification du projet **7**

9 Livrables et critères de réussite **7**

1 Introduction et contexte du projet

Dans un contexte où les organisations cherchent à accélérer le développement et le déploiement de leurs applications, l'adoption des pratiques DevOps et des technologies cloud est devenue essentielle. Toutefois, cette automatisation soulève des enjeux importants en matière de sécurité, de gestion des risques et de conformité.

L'approche DevSecOps répond à ces défis en intégrant les contrôles de sécurité tout au long du cycle de vie des applications. Néanmoins, la mise en place d'une chaîne CI/CD sécurisée ne garantit pas, à elle seule, une maîtrise suffisante des risques ni une traçabilité complète des mesures de protection appliquées.

C'est dans cette perspective que s'inscrit notre projet, qui vise à **concevoir et mettre en œuvre une infrastructure DevSecOps sur Kubernetes, intégrant une démarche de Gouvernance, Gestion des Risques et Conformité (GRC)**.

L'objectif est de combiner l'automatisation technique des processus de développement, de sécurisation et de déploiement avec une approche structurée de gestion des risques, de définition des contrôles et de vérification de leur efficacité.

Problématique

Comment automatiser et sécuriser le cycle de vie d'une application déployée sur Kubernetes, tout en intégrant les exigences de gouvernance, de gestion des risques et de conformité, et en assurant la traçabilité des contrôles de sécurité mis en œuvre ?

Solution proposée

Pour répondre à cette problématique, le projet repose sur le développement d'une application web de gestion de tâches, organisée en trois couches : **Next.js** pour le frontend, **Python (FastAPI)** pour le backend, réparti en deux microservices (Auth Service et Task Service), et **MongoDB** pour la gestion des données.

Cette application servira de support expérimental pour la mise en œuvre d'un cycle DevSecOps complet, comprenant l'intégration continue, les analyses de sécurité, la conteneurisation, le déploiement automatisé et la supervision.

L'infrastructure AWS, comprenant le cluster **EKS**, sera provisionnée à l'aide de **Terraform**. Les processus CI/CD s'appuieront sur **GitHub Actions**, les outils d'analyse de sécurité et **Argo CD** pour le déploiement GitOps.

En parallèle, une démarche GRC sera intégrée aux différentes étapes du projet afin d'identifier les risques, de sélectionner les mesures de sécurité adaptées et de documenter les résultats des contrôles.

Objectifs principaux

— Développer et déployer une application trois tiers sur un cluster Kubernetes AWS EKS.
— Automatiser les processus de test, d'analyse de sécurité, de construction et de déploiement des applications.
— Mettre en œuvre des mécanismes de sécurisation de l'infrastructure, des accès, des communications et des données.
— Intégrer une démarche GRC permettant d'identifier, d'évaluer et de traiter les risques associés à la plateforme.
— Définir et évaluer une sélection de 15 à 20 contrôles de sécurité, en associant chaque contrôle à des preuves vérifiables.
— Comparer l'état initial et l'état après sécurisation afin de mesurer les améliorations apportées.

Périmètre du projet

Le projet se concentre sur un environnement de démonstration comprenant une application, un cluster Kubernetes et une chaîne DevSecOps. La démarche GRC s'appuiera notamment sur les référentiels NIST CSF 2.0, ISO/IEC 27001 :2022 et CIS Benchmark EKS, à travers une sélection de contrôles adaptés au périmètre technique.

Le projet ne vise pas une certification ISO 27001 ni une conformité réglementaire exhaustive, mais la démonstration d'une intégration cohérente entre les pratiques DevSecOps et la gestion des risques de cybersécurité.

2 L'application : un gestionnaire de tâches très simple

Besoins fonctionnels

— **F1** – Créer un compte et se connecter.
— **F2** – Créer une tâche (titre, description, échéance).
— **F3** – Consulter la liste, filtrer par statut.
— **F4** – Modifier une tâche et changer son statut (à faire, en cours, terminée).
— **F5** – Supprimer une tâche.

Besoins non fonctionnels

— **N1** – Accès en HTTPS uniquement.
— **N2** – Chaque utilisateur ne voit que ses tâches.
— **N3** – Aucun secret dans Git.
— **N4** – Déploiement automatisé et reproductible.
— **N5** – Sauvegarde MongoDB restaurable.
— **N6** – Supervision de base (métriques, tableaux de bord).

2.1 Diagramme pieuvre

Le diagramme pieuvre présente les interactions entre la plateforme DevSecOps sur Kubernetes et les différents éléments de son environnement. Il met en évidence la fonction principale (FP), qui consiste à automatiser, sécuriser et superviser le déploiement d'une application web tout en intégrant une démarche GRC. Les fonctions contraintes (FC1 à FC10) représentent les exigences techniques, fonctionnelles et de sécurité à respecter.

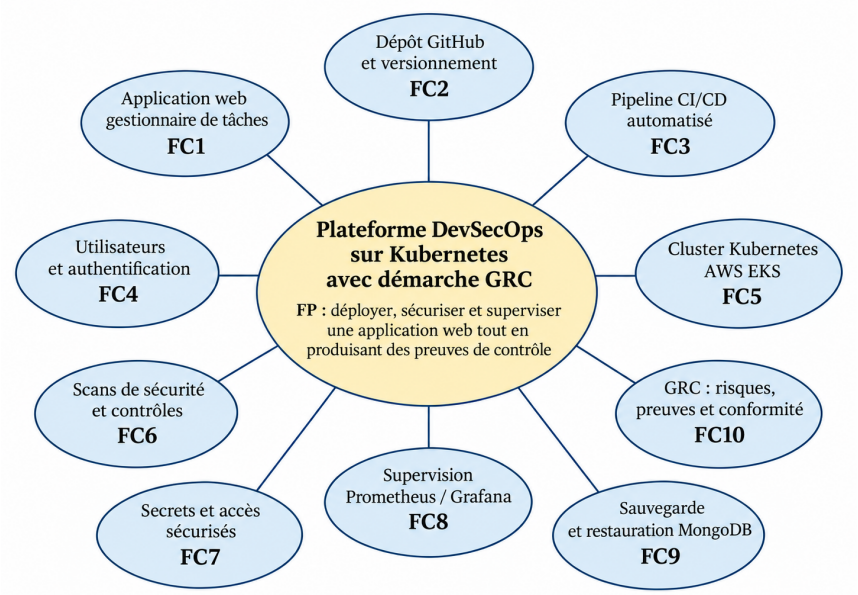


Figure 1 – Diagramme pieuvre de la plateforme DevSecOps sur Kubernetes avec démarche GRC

Le tableau 1 synthétise la fonction principale et les fonctions contraintes identifiées dans le diagramme, ainsi que leurs rôles dans le fonctionnement de la plateforme.

Tableau 1 – Fonction principale et fonctions contraintes du système

Code	Fonction	Interprétation
FP	Déployer, sécuriser et superviser l'application	Automatiser le déploiement et la supervision, appliquer les contrôles de sécurité et conserver les preuves GRC.
FC1	Application web	Fournir un gestionnaire de tâches fonctionnel.
FC2	Gestion du code source	Versionner et gérer les modifications avec GitHub.
FC3	Pipeline CI/CD	Automatiser les tests, les scans et la livraison.
FC4	Authentification	Gérer les comptes et sécuriser les accès utilisateurs.
FC5	Kubernetes / AWS EKS	Orchestrer les conteneurs et héberger l'application.
FC6	Contrôles de sécurité	Détecter les vulnérabilités et bloquer les versions non conformes.
FC7	Gestion des secrets	Protéger les identifiants et limiter les permissions.
FC8	Supervision	Surveiller l'application avec Prometheus et Grafana.
FC9	Sauvegarde MongoDB	Assurer la sauvegarde et la restauration des données.
FC10	Démarche GRC	Identifier les risques, évaluer les contrôles et conserver les preuves.

3 Architecture globale

L'architecture proposée repose sur une approche **DevSecOps intégrant les pratiques de Gouvernance, Gestion des Risques et Conformité (GRC)** tout au long du cycle de vie de l'application.

Le processus débute par le développement collaboratif sur GitHub. Chaque modification déclenche une chaîne CI/CD avec GitHub Actions, intégrant des tests automatisés et des analyses de sécurité à l'aide de SonarQube et Trivy. Les images Docker validées sont ensuite publiées dans AWS ECR.

L'infrastructure cloud est provisionnée avec Terraform sur AWS, tandis qu'Argo CD assure le déploiement continu de l'application sur un cluster Kubernetes EKS selon l'approche GitOps.

L'application repose sur une architecture trois tiers : **Next.js (frontend), Python avec FastAPI (backend) et MongoDB (base de données)**. Prometheus et Grafana permettent de superviser l'environnement.

La démarche **GRC** intervient transversalement pour encadrer les décisions de sécurité, identifier les risques, définir les contrôles applicables et conserver les preuves nécessaires à leur évaluation.

La figure ci-dessous présente l'architecture générale de la solution et les interactions entre ses principaux composants. Le dépôt GitOps contient les références des images validées, qui sont récupérées depuis ECR lors du déploiement sur EKS.

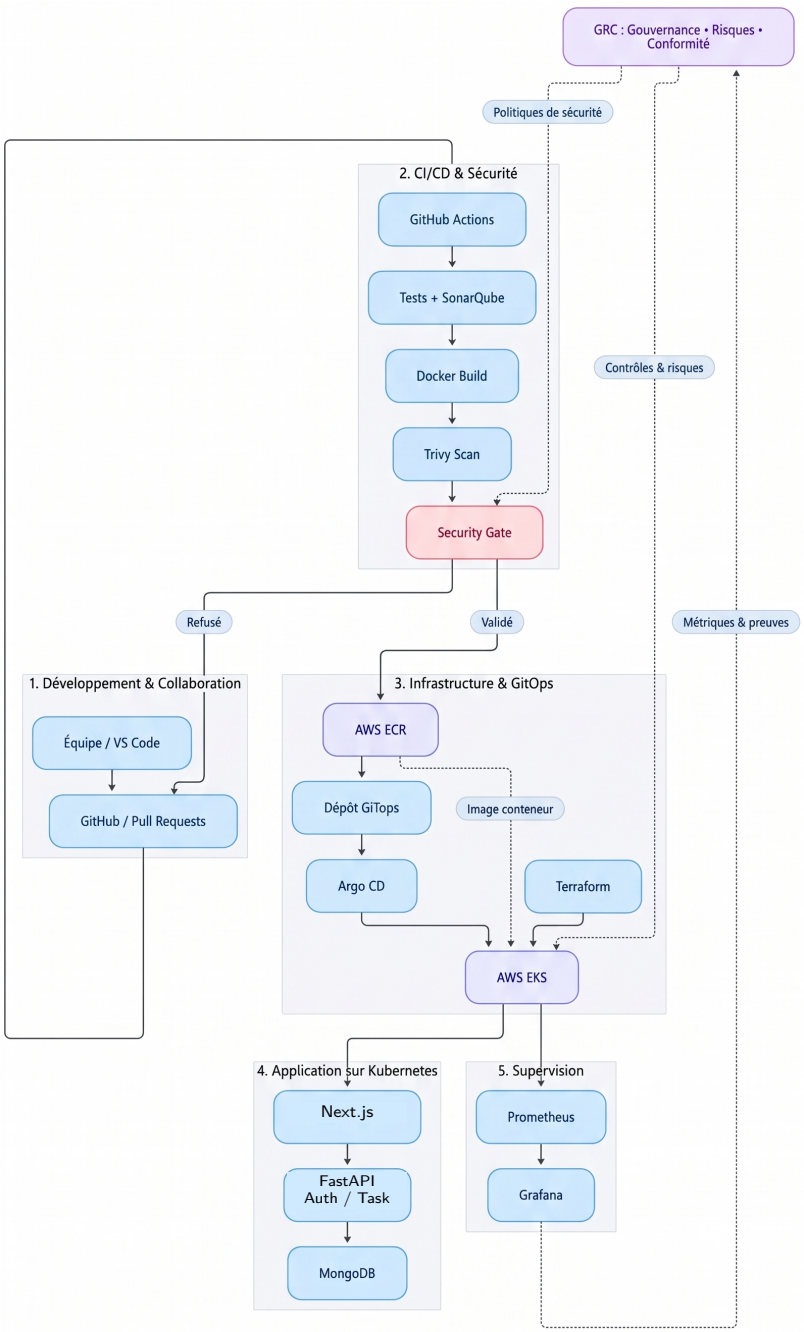


Figure 2 – Architecture globale de la plateforme DevSecOps intégrant une démarche GRC

4 Diagrammes UML

Le backend est découpé en **deux microservices FastAPI** : **Auth Service** (comptes, jetons JWT) et **Task Service** (tâches). Chacun a son code, son image Docker, son déploiement et sa base logique MongoDB. Les appels du frontend aux API passent par l'*Ingress HTTPS*, qui assure le routage vers les deux services.

4.1 Cas d'utilisation

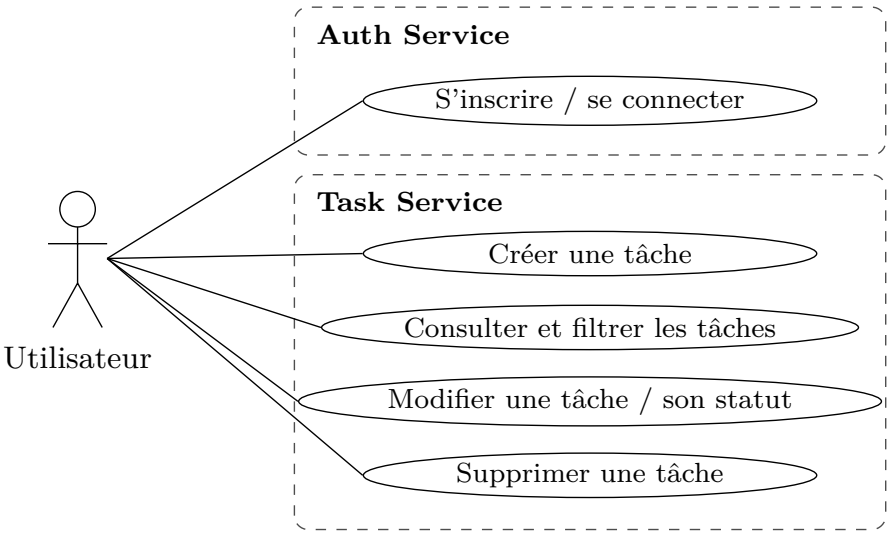


Figure 3 – Un seul acteur, deux microservices

4.2 Diagramme de composants

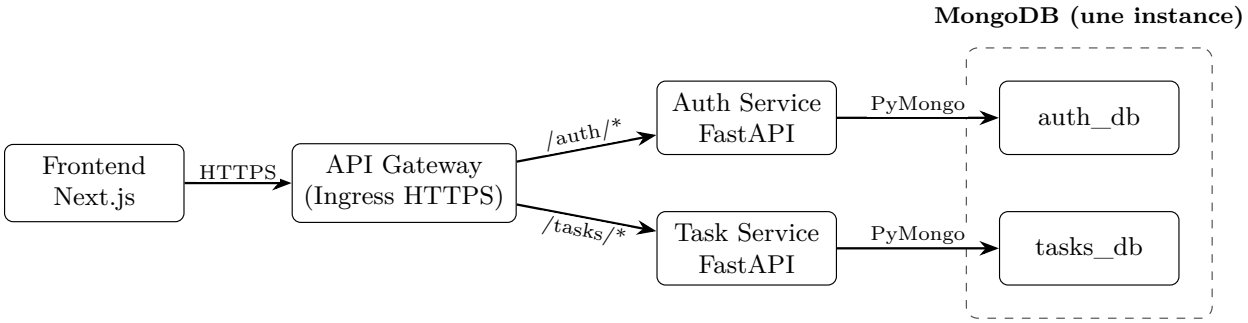


Figure 4 – Composants (Task Service vérifie le JWT localement)

4.3 Diagramme de classes

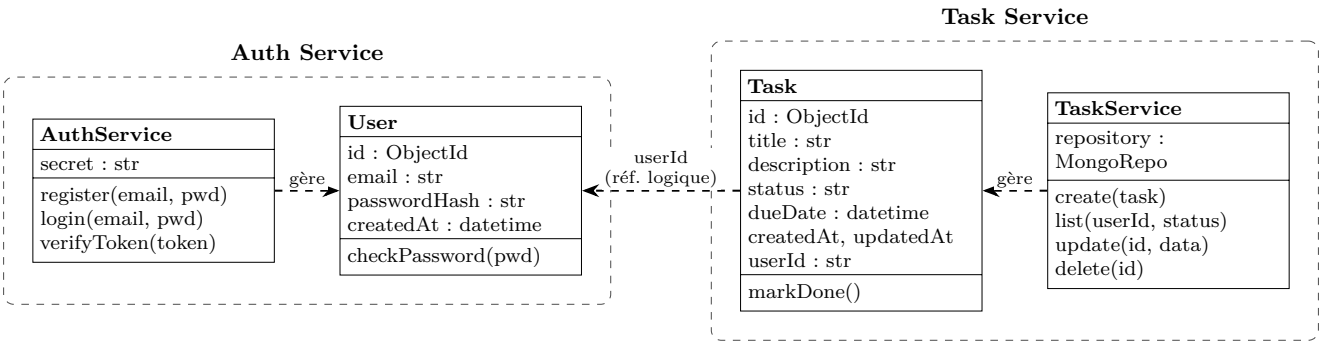


Figure 5 – Classes de chaque microservice

4.4 Diagramme de séquence : ajouter une tâche

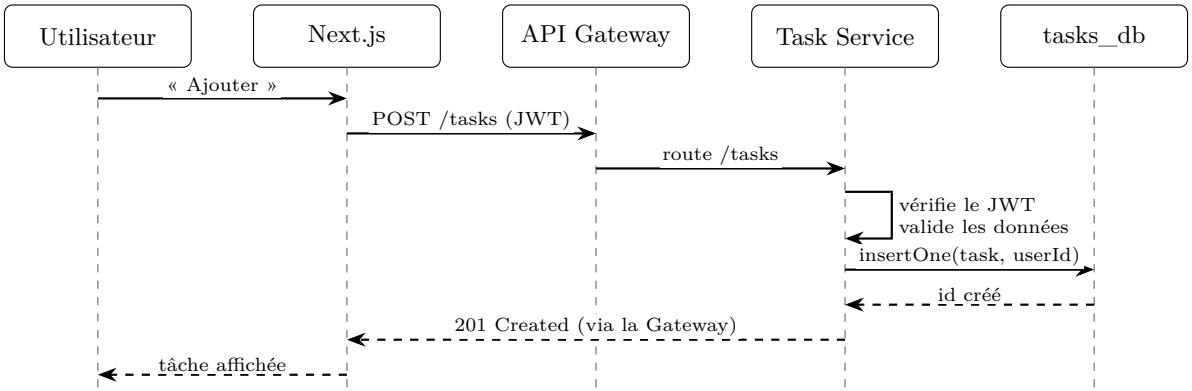


Figure 6 – Création d'une tâche (le JWT a été obtenu auparavant via `POST /auth/login`)

5 Architecture de la base de données

Chaque microservice possède **sa propre base logique, sur une instance MongoDB commune** : Auth Service ne lit que `auth_db`, Task Service que `tasks_db`. Aucun service ne va lire dans la base de l'autre.

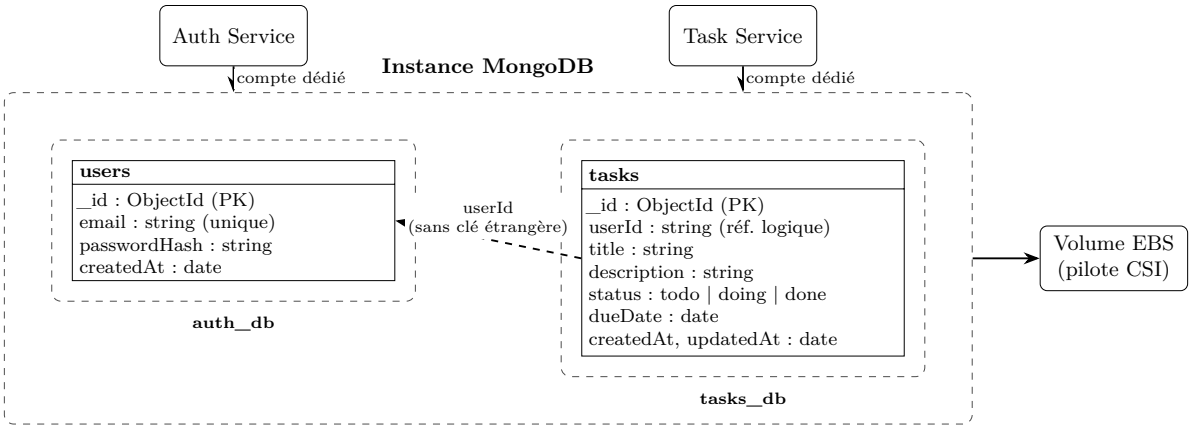


Figure 7 – Une base par microservice, sur une seule instance MongoDB

— **Pas de jointure entre services :** une tâche garde seulement le `userId` tiré du jeton JWT. Task Service n'a jamais besoin d'interroger Auth Service.
— **Moindre privilège :** chaque service utilise un compte MongoDB limité à sa base, et les NetworkPolicies n'autorisent que ces deux services à joindre MongoDB.
— **Une seule instance :** pour rester simple et peu coûteux (la haute disponibilité avancée est hors périmètre). Les deux bases pourront être hébergées sur des instances distinctes en adaptant les paramètres de connexion.
— **Index :** `users.email` (unique) ; `tasks.{userId, status}` pour le filtre de la liste.
— **Persistance et sauvegarde :** volume EBS via le pilote CSI ; export régulier des deux bases, restauration testée et chronométrée.

6 Outils utilisés

Bloc	Outils	Rôle
Frontend	Next.js	Interface web de gestion des tâches
Backend	Python, FastAPI, PyMongo	API REST, validation des données, authentification JWT
Base de données	MongoDB, volume EBS (CSI)	Stockage persistant des comptes et des tâches
Code	GitHub, pull requests	Historique, revue croisée, dépôts applicatif et GitOps
Infrastructure	Terraform, AWS EKS, IAM, VPC	Création reproductible du cluster et du réseau
CI	GitHub Actions, Docker, AWS ECR	Tester, construire, publier les images validées
Analyses	SonarQube, Trivy, Dependency-Check	Code, images, dépendances (pip-audit en complément si besoin)
Déploiement	Kubernetes, Argo CD	Synchronisation du cluster avec Git (GitOps)
Durcissement	RBAC, NetworkPolicies, Pod Security Admission	Droits, flux réseau, privilèges des pods
Supervision	Helm, Prometheus, Grafana	Installation, métriques, tableaux de bord
GRC	Registre et grille versionnés	Risques, décisions, contrôles et preuves

7 Sécurité et démarche GRC

Nous suivons **NIST CSF 2.0** (gouverner, identifier, protéger, détecter, répondre, rétablir), nous choisissons des vérifications techniques dans le **CIS Benchmark EKS**, et nous nous inspirons d'**ISO/IEC 27001 :2022** pour les responsabilités. Chaque contrôle de la grille précise : référence, périmètre, responsable, procédure de vérification, résultat et preuve.

Le tableau suivant présente des exemples de risques à intégrer au registre de 8 à 10 risques.

Risque	Mesure	Preuve attendue
Droits AWS excessifs du CI	OIDC + rôle IAM limité	Politique IAM, action interdite refusée
Image vulnérable livrée	Seuils de blocage + scan avant publication	Rapport Trivy, pipeline bloqué
Interfaces d'administration exposées	Sources autorisées restreintes	Règles réseau, tests d'accès
Pod trop privilégié	Pod Security Admission	Manifeste non conforme refusé
Accès inutile à MongoDB	NetworkPolicies	Accès autorisé depuis Auth Service et Task Service, refusé ailleurs
Perte de données	Volume persistant, sauvegarde	Restauration réussie, durée mesurée

Gouvernance minimale : responsables désignés (infrastructure, pipeline, registre des risques), revue croisée des changements sensibles, exceptions avec motif, responsable, date d'expiration et mesure compensatoire. **Statuts d'un contrôle :** satisfait, partiellement satisfait, non satisfait, non applicable, non vérifié.

8 Planification du projet

La réalisation du projet est organisée sur une période prévisionnelle de huit semaines. La première semaine est consacrée au cadrage, à la conception de l'architecture, à l'organisation du travail sur Jira et GitHub, ainsi qu'à l'identification des risques initiaux.

Les semaines suivantes couvrent le développement, le provisionnement de l'infrastructure, la mise en place des pipelines DevSecOps, le déploiement GitOps et la sécurisation de Kubernetes.

Les activités GRC et la documentation sont menées de manière transversale tout au long du projet.

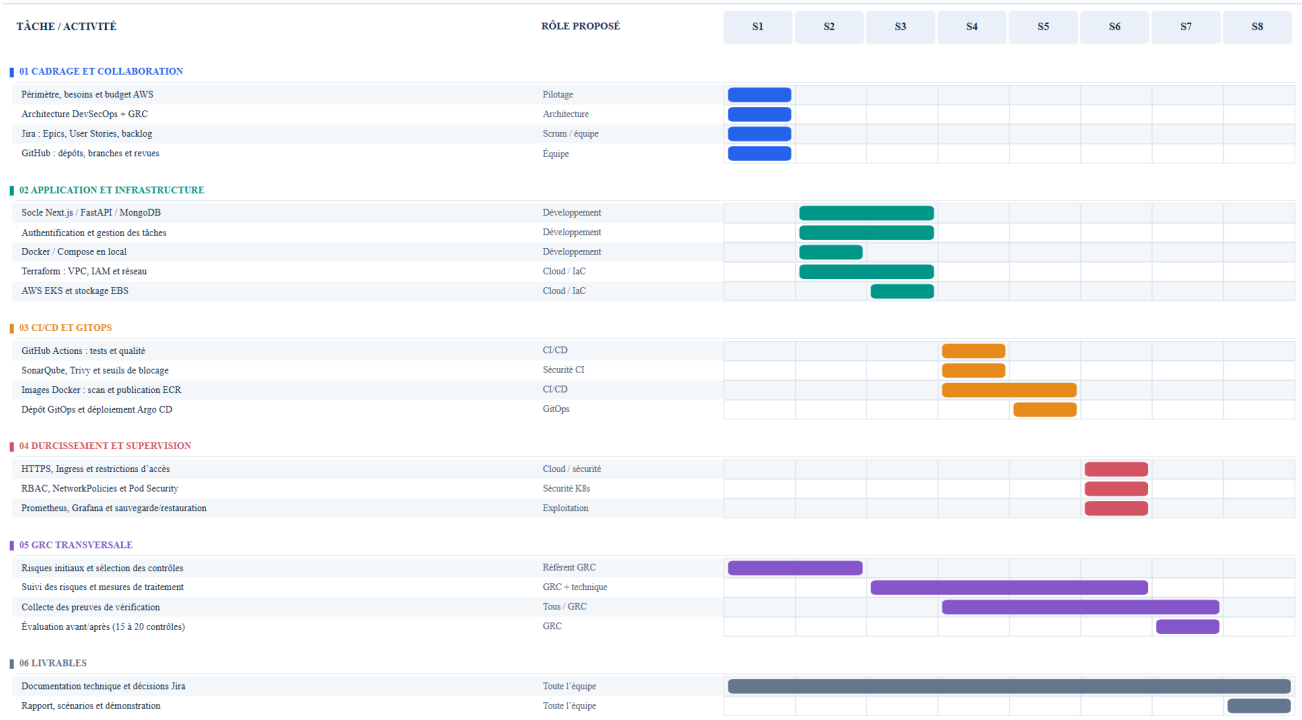


Figure 8 – Diagramme de Gantt prévisionnel du projet DevSecOps et GRC

9 Livrables et critères de réussite

Livrables

- Application de gestion de tâches fonctionnelle (Next.js, FastAPI et MongoDB)
- Code, fichiers Terraform, Docker, manifestes Kubernetes et workflows CI/CD versionnés
- Schémas d'architecture et guides de déploiement, de configuration et de suppression
- Registre de 8 à 10 risques et matrice de 15 à 20 contrôles avec plans de traitement
- Rapports de tests et d'analyses, preuves de contrôle et comparaison avant/après
- Rapport final et support de soutenance

Critères de réussite

- Une modification validée est déployée automatiquement via CI/CD et GitOps.
- Une image qui dépasse les seuils de vulnérabilités est bloquée avant son déploiement.
- Les tests confirment la limitation des accès et la segmentation réseau attendues.
- Une sauvegarde permet de restaurer les données de test.
- Chaque contrôle évalué possède un statut, une justification et une preuve associée lorsqu'il est satisfait.
- L'évaluation avant/après fait ressortir les améliorations obtenues et les risques résiduels.